Chapter 5: CPU Scheduling 1

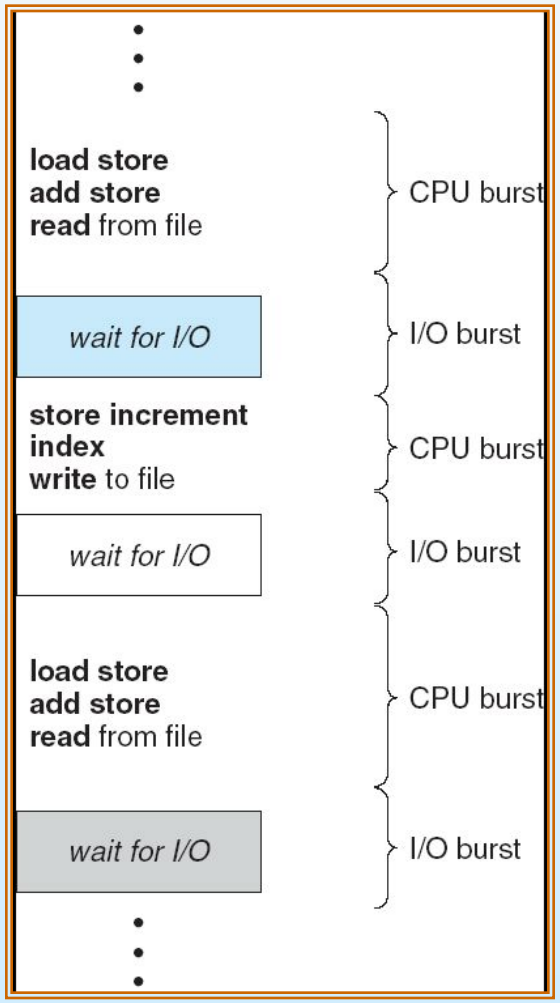
Basic Concepts

- Maximum CPU utilization obtained with multiprogramming
- CPU–I/O Burst Cycle – Process execution consists of a *cycle* of CPU execution and I/O wait
- CPU burst distribution

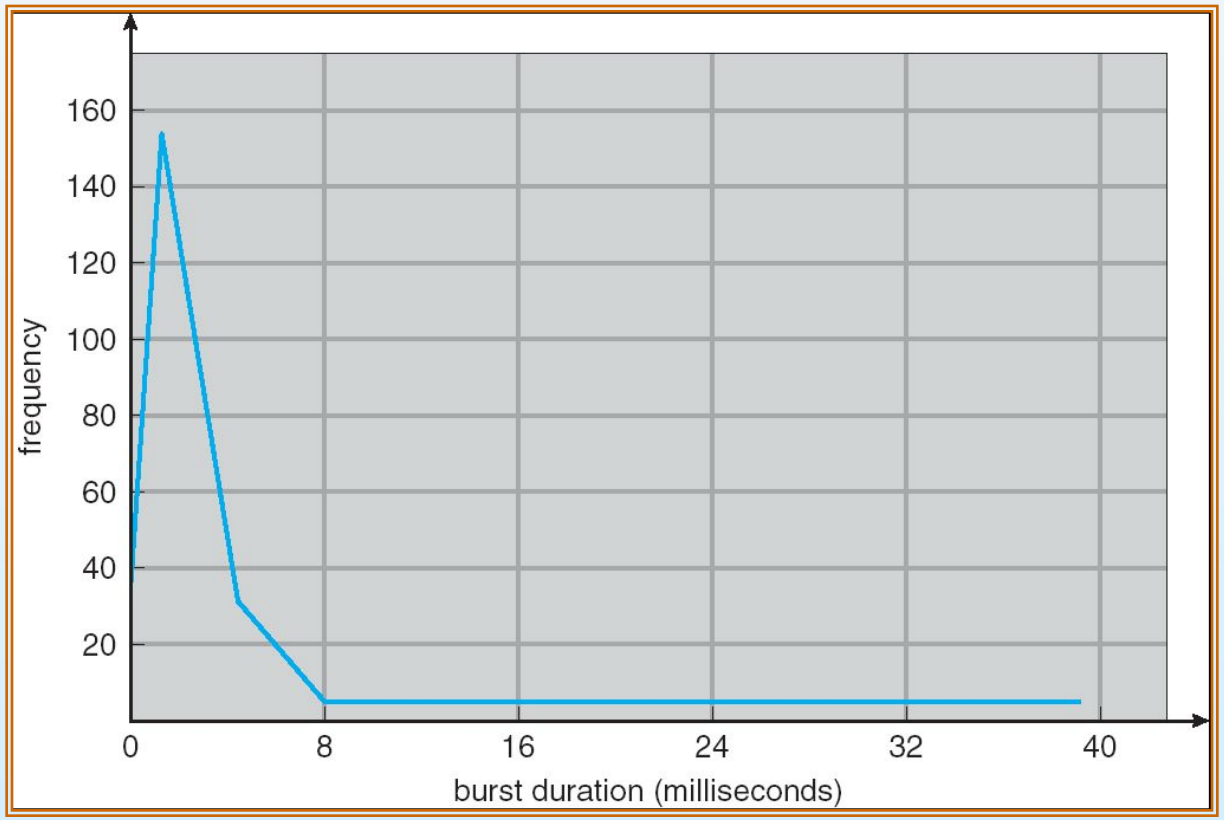
Chapter 5: CPU Scheduling

- Basic Concepts
- Scheduling Criteria
- Scheduling Algorithms
- Multiple-Processor Scheduling
- Real-Time Scheduling
- Thread Scheduling
- Operating Systems Examples
- Java Thread Scheduling
- Algorithm Evaluation

Alternating Sequence of CPU And I/O Bursts



Histogram of CPU-burst Times



CPU Scheduler

- Selects from among the processes in memory that are ready to execute, and allocates the CPU to one of them
- CPU scheduling decisions may take place when a process:
  1. Switches from running to waiting state
  2. Switches from running to ready state
  3. Switches from waiting to ready
  4. Terminates
- Scheduling under 1 and 4 is *nonpreemptive*
- All other scheduling is *preemptive*

Dispatcher

- Dispatcher module gives control of the CPU to the process selected by the short-term scheduler; this involves:
  - switching context
  - switching to user mode
  - jumping to the proper location in the user program to restart that program
- *Dispatch latency* – time it takes for the dispatcher to stop one process and start another running

Scheduling Criteria

- CPU utilization – keep the CPU as busy as possible
- **Throughput** – # of processes that complete their execution per time unit
- **Turnaround** time – amount of time to execute a particular process
- **Waiting** time – amount of time a process has been waiting in the ready queue
- **Response** time – amount of time it takes from when a request was submitted until the first response is produced, **not** output (for time-sharing environment)

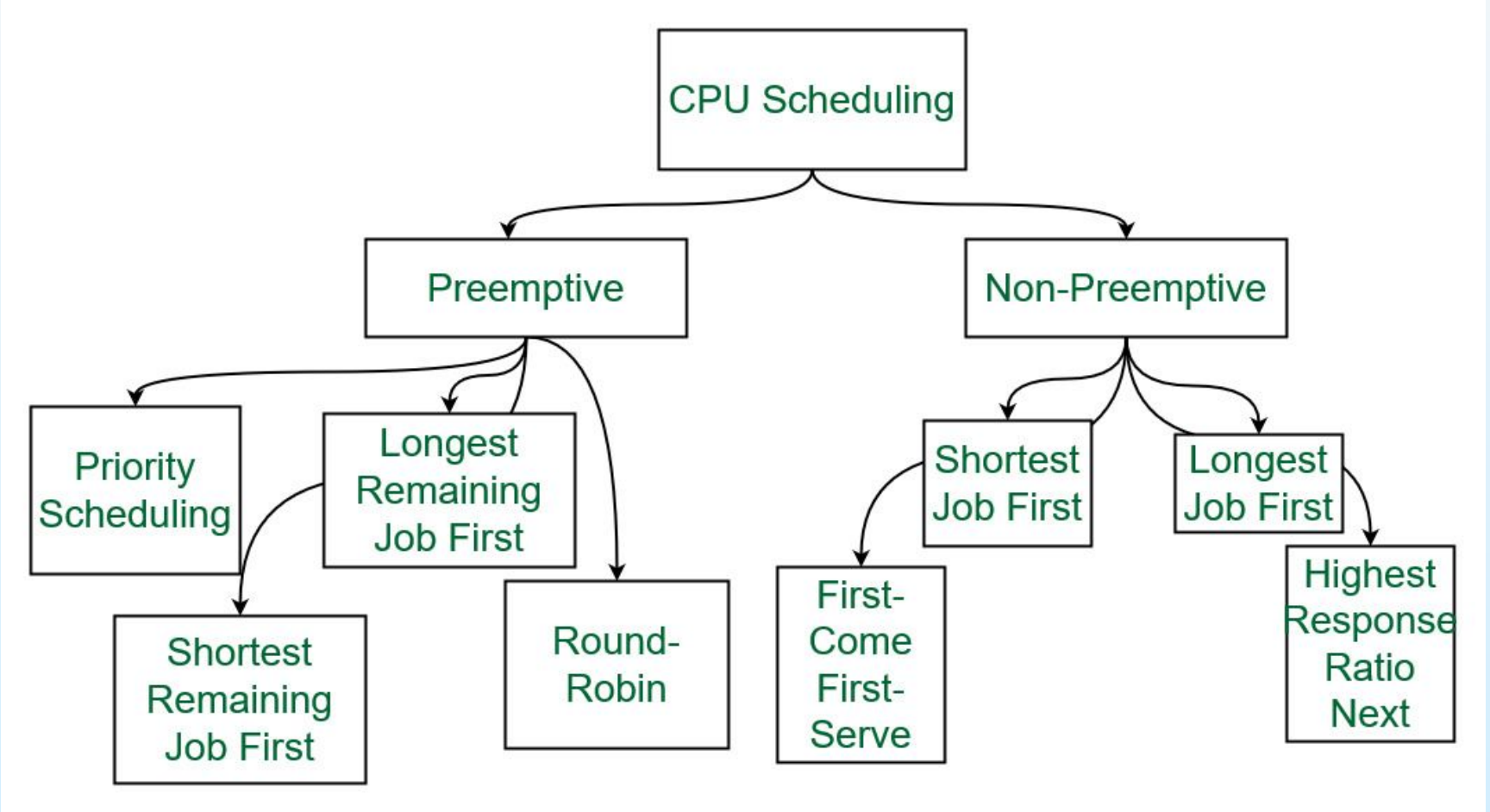
Optimization Criteria

- Max CPU utilization
- Max throughput
- Min turnaround time
- Min waiting time
- Min response time

Terminologies Used in CPU Scheduling

- **Arrival Time:** Time at which the process arrives in the ready queue.
- **Completion Time:** Time at which process completes its execution.
- **Burst Time:** Time required by a process for CPU execution.
- **Turn Around Time:** Time Difference between completion time and arrival time.
  - Turn Around Time = Completion Time – Arrival Time
- **Waiting Time(W.T):** Time Difference between turn around time and burst time.
  - Waiting Time = Turn Around Time – Burst Time

Different Types of CPU Scheduling Algorithms

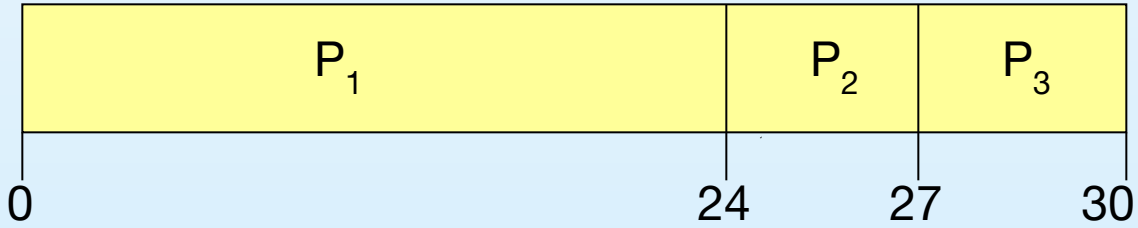


Algorithm	Allocation is	Complexity	Average waiting time (AWT)	Preemption	Starvation	Performance
FCFS	According to the arrival time of the processes, the CPU is allocated.	Simple and easy to implement	Large.	No	No	Slow performance
SJF	Based on the lowest CPU burst time (BT).	More complex than FCFS	Smaller than FCFS	No	Yes	Minimum Average Waiting Time
LJFS	Based on the highest CPU burst time (BT)	More complex than FCFS	Depending on some measures e.g., arrival time, process size, etc.	No	Yes	Big turn-around time
LRTF	Same as LJFS the allocation of the CPU is based on the highest CPU burst time (BT). But it is preemptive	More complex than FCFS	Depending on some measures e.g., arrival time, process size, etc.	Yes	Yes	The preference is given to the longer jobs
SRTF	Same as SJF the allocation of the CPU is based on the lowest CPU burst time (BT). But it is preemptive.	More complex than FCFS	Depending on some measures e.g., arrival time, process size, etc	Yes	Yes	The preference is given to the short jobs
RR	According to the order of the process arrives with fixed time quantum (TQ)	The complexity depends on Time Quantum size	Large as compared to SJF and Priority scheduling.	Yes	No	Each process has given a fairly fixed time
Priority Pre-emptive	According to the priority. The bigger priority task executes first	This type is less complex	Smaller than FCFS	Yes	Yes	Well performance but contain a starvation problem
Priority non-preemptive	According to the priority with monitoring the new incoming higher priority jobs	This type is less complex than Priority preemptive	Preemptive Smaller than FCFS	No	Yes	Most beneficial with batch systems
MLQ	According to the process that resides in the bigger queue priority	More complex than the priority scheduling algorithms	Smaller than FCFS	No	Yes	Good performance but contain a starvation problem
		It is the most Complex but				

First-Come, First-Served (FCFS) Scheduling

Process	Burst Time
P_1	24
P_2	3
P_3	3

- Suppose that the processes arrive in the order: P_1 , P_2 , P_3 The Gantt Chart for the schedule is:



- TT=Completion Time- Arrival Time/ wt+bt
- Waiting time for WT=TT-BT/
- P_1 = 0; P_2 = 24; P_3 = 27
- Average waiting time: (0 + 24 + 27)/3 = 17

FCFS Scheduling (Cont.)

Suppose that the processes arrive in the order

P_2 , P_3 , P_1

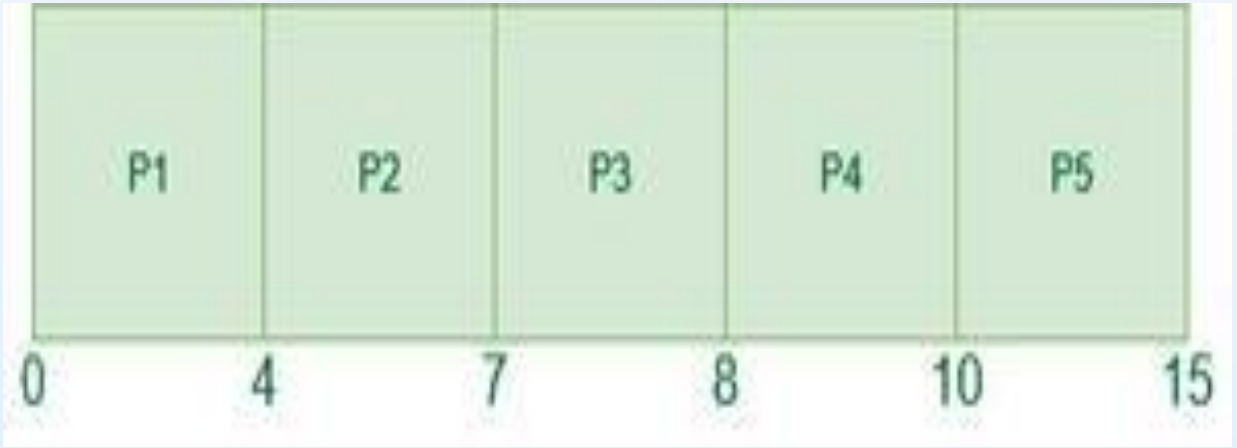
- The Gantt chart for the schedule is:

- Waiting time for $P_1 = 6$; $P_2 = 0$; $P_3 = 3$
- Average waiting time: (6 + 0 + 3)/3 = 3
- Much better than previous case
- *Convoy effect* short process behind long process

Practice Example.

Processes	Arrival Time	Burst Time
P1	0	4
P2	1	3
P3	2	1
P4	3	2
P5	4	5

Gantt chart and wait time



P1 = 0 – 0 = 0
P2 = 4 – 1 = 3
P3 = 7 – 2 = 5
P4 = 8 – 3 = 5
P5 = 10 – 4 = 6

Pros & Cons of FCFS.

Pros:

- FCFS is the easiest CPU scheduling algorithm to understand and implement because it uses a simple queue (FIFO).
- Processes are executed in the order they arrive, ensuring fairness with no favoritism.
- Since there is no complex decision-making or frequent context switching, CPU overhead is minimal
- Works well in batch processing environments where response time is not a major concern.

Cons

- **Convoy Effect:** A long CPU-bound process can delay all shorter processes waiting behind it, reducing overall system performance.
- **High Average Waiting Time:** Short processes may wait for a long time if they arrive after a lengthy process.
- **Non-Preemptive:** Once a process starts executing, it cannot be interrupted, even if a higher-priority or shorter process arrives.
- **Not Suitable for Time-Sharing Systems:** FCFS performs poorly in systems requiring quick responses, such as interactive or real-time applications.

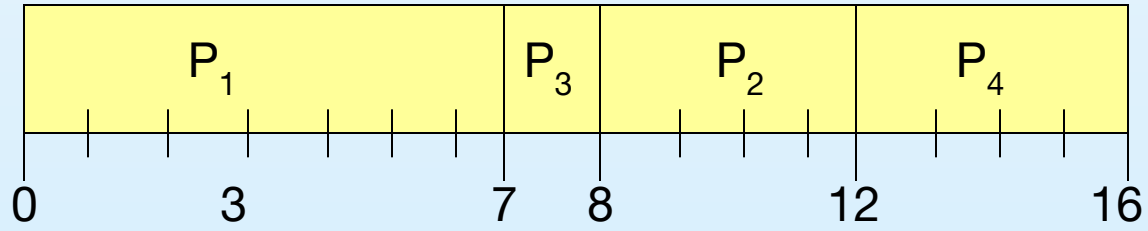
Shortest-Job-First (SJF) Scheduling

- Associate with each process the length of its next CPU burst. Use these lengths to schedule the process with the shortest time
- Two schemes:
  - nonpreemptive – once CPU given to the process it cannot be preempted until completes its CPU burst
  - preemptive – if a new process arrives with CPU burst length less than remaining time of current executing process, preempt. This scheme is know as the Shortest-Remaining-Time-First (SRTF)
- SJF is optimal – gives minimum average waiting time for a given set of processes

Example of Non-Preemptive SJF

Process	Arrival Time	Burst Time
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

- SJF (non-preemptive)



- Average waiting time = (0 + 6 + 3 + 7)/4 = 4